

# Systems Programming & OS Master Study Guide: Milestone 1 & Beyond

## PART 1: Core OS Theory & Systems Architecture

### 1. Low-Level Prerequisites & Environment Setup

- **Kernel:** The core, low-level program of an operating system that maintains absolute control over all hardware resources in the system, including the CPU, physical memory (RAM), and I/O devices.
- **System Calls:** When user-level programs need to touch hardware directly, create memory spaces, or spawn processes, they must request the kernel to do so via system calls. User programs run in a restricted User Mode; system calls act as the bridge to Kernel Mode, where the kernel executes the privileged task and returns the result back to user space.
- **POSIX (Portable Operating System Interface):** A standardized set of rules and C function signatures (such as `fork()` , `wait()` ) acting as a universal contract. Any OS conforming to POSIX provides these exact function definitions so developers can write portable software across Unix, Linux, and macOS without rewriting code.
- **WSL (Windows Subsystem for Linux):** Windows uses a proprietary kernel API, and standard Windows compilers lack POSIX header definitions. Trying to include `<sys/wait.h>` directly on Windows results in a “cannot open source file” error because Windows natively cannot execute or interpret Linux kernel system calls. WSL provides a lightweight Linux kernel instance directly inside Windows alongside the GNU Compiler Collection (gcc) and standard package managers (apt).
- **Compilation & Libraries:** Header files ( `.h` ) provide function prototypes (declarations) used by the compiler to detect type mismatches and interface errors. The compiled implementation lives in binary library files ( `.lib` / `.a` / `.so` ). Compilers themselves are regular user-space programs, not part of the OS kernel, so running a compiler does not require kernel-mode switching.

### 2. Process Management & Memory Architecture

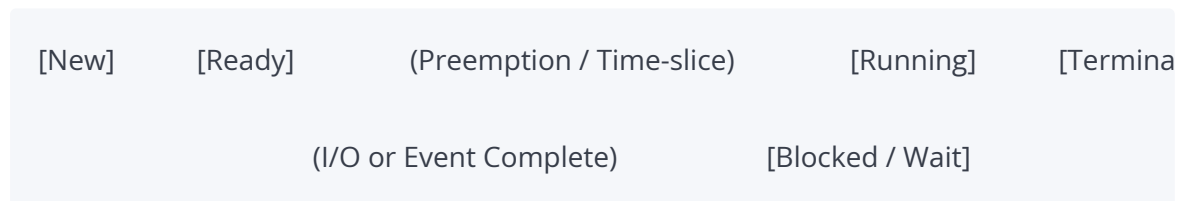
#### Program vs. Process

- **Program:** A passive entity consisting of compiled machine instructions and data residing statically on disk (e.g., ELF binary or `.exe`). It consumes no CPU cycles or RAM.
- **Process:** An active program in execution instantiated in RAM. It actively utilizes CPU registers, execution threads, memory pages, and OS resources.

## Process Operations

- **Create():** Allocates system resources, sets up the address space, and initializes the PCB.
- **Schedule():** The act of the CPU scheduler selecting an eligible process from the Ready queue to execute on a CPU core.
- **Execute() / Run():** Executing machine instructions fetched from the Text segment.
- **Block() / Wait():** Transitions a process to the Waiting/Blocked state when executing an I/O operation or blocking system call.
- **Suspend():** Moves a process from main memory to disk (swapping).
- **Resume():** Moves a suspended process from disk back into main memory.
- **Terminate():** Deallocates all resources, releases memory, and closes file descriptors.

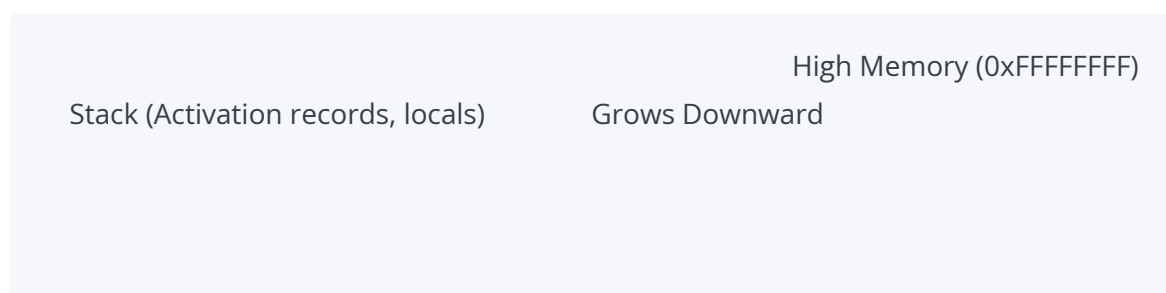
## Process State Transitions



- **Ready → Running:** Selected by the CPU Scheduler and dispatched.
- **Running → Ready:** Preempted due to time-slice expiration or the arrival of a higher-priority task.
- **Running → Blocked:** Process issues an I/O request or blocking system call.
- **Blocked → Ready:** The requested I/O operation completes.

## Memory Layout in RAM

A running process occupies four virtual memory segments:



Heap (Dynamic memory via malloc())

Grows Upward

Data Segment

- Initialized: Predefined globals
- BSS (Uninitialized): Zeroed globals

Text Segment (Compiled machine code)

Read-Only

Low Memory (0x00000000)

- **Text Segment:** Holds read-only machine code instructions to prevent self-modifying code vulnerabilities.
- **Data Segment:** Holds global and static variables. Initialized data holds explicitly assigned values (e.g., `int x = 10;`); Uninitialized Data (BSS) holds variables zero-initialized by default.
- **Heap Segment:** Dynamically allocated memory at runtime via `malloc()` in C; grows upward toward higher memory addresses.
- **Stack Segment:** Holds function call activation records, local function variables, and return instruction pointers; grows downward toward lower memory addresses.

### Process Control Block (PCB)

The kernel data structure serving as the process's internal identity record. It stores:

- **Process ID (PID) & Parent PID (PPID):** Unique numerical identifiers.
- **Process State:** Current execution phase (New, Ready, Running, Blocked, Terminated).
- **Program Counter (PC):** Memory address of the next instruction to execute.
- **CPU Registers & Context:** Accumulators, index registers, and stack pointers preserved during a context switch.
- **Memory Management Information:** Base/limit registers and page table pointers.
- **I/O Status Information:** Open file descriptor tables and allocated devices.

## 3. Concurrency, Critical Sections & Synchronization

### The Concurrency Problem & Race Conditions

- **Inter-Process Communication (IPC):** When concurrent processes exchange data through a shared medium (such as shared memory or pipes).
- **Synchronization:** Coordinating concurrent processes to ensure correct execution order and prevent data corruption when accessing shared resources.

- **Race Condition:** A flaw where multiple execution threads access and modify shared data simultaneously, producing final results that depend entirely on arbitrary CPU scheduling and execution timing.

**Low-Level Cause:** A high-level operation like `count++` consists of three distinct, non-atomic CPU instructions:

```
LOAD R1, count
INCREMENT R1
STORE count, R1
```

If a context switch occurs between `LOAD` and `STORE`, the updates from one thread overwrite the other silently.

### Synchronization Types & Pitfalls

- **Competitive Synchronization:** Multiple processes compete to access a single shared resource; failures lead to Inconsistency or Data Loss.
- **Cooperative Synchronization:** The execution of one process directly affects or depends on the progress of another; failures lead to Deadlock (processes permanently blocked waiting for an event that will never happen).

### The Critical Section Problem

The Critical Section (CS) is the region of code accessing shared resources (global variables, files, tables). A correct synchronization solution must satisfy three criteria:

1. **Mutual Exclusion:** If process  $P_i$  is executing in its critical section, no other processes can enter their critical sections.
2. **Progress:** If no process is in its critical section and some wish to enter, only processes not in their remainder sections can participate in deciding who enters next.
3. **Bounded Waiting:** A bound must exist on the number of times other processes can enter their critical sections after a process requests entry, preventing starvation.

### Synchronization Primitives

- **Lock Variables:** Software flags (`lock = 0` for free, `1` for busy); broken if testing and setting are non-atomic.
- **Test-and-Set Lock (TSL):** A hardware-level atomic instruction that checks and sets a lock in a single clock cycle without preemption.

- **Mutex (Mutual Exclusion Lock):** A binary locking primitive ( `acquire()` / `release()` ) used to protect a critical section.
- **Semaphores & Condition Variables:** Advanced signaling and resource-counting synchronization tools.

## 4. Threads vs. Processes

| Feature           | Process (Heavyweight)                                                     | Thread (Lightweight)                                                |
|-------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------|
| Address Space     | Completely isolated virtual memory space                                  | Shares address space with peer threads                              |
| Resource Overhead | High creation/destruction overhead (allocates PCB, page tables, FD table) | Low overhead (reuses parent process resources)                      |
| Shared Resources  | None by default (requires explicit IPC / pipes / shared memory)           | Shares Code (Text), Data (Globals), Heap, and Open Files            |
| Private Data      | Entire address space is private                                           | Private Thread ID (TID), Program Counter (PC), Registers, and Stack |
| Context Switching | Slow (flushes CPU caches, switches MMU page tables)                       | Fast (switches execution registers and stack pointer only)          |

- **User-Level Threads (ULT):** Managed in user space by runtime libraries. Fast switching without kernel transitions, but if one thread performs a blocking system call, the kernel blocks the entire process.
- **Kernel-Level Threads (KLT):** Scheduled directly by the OS kernel. Supports true multi-core hardware parallelism; blocking one thread leaves sibling threads running, but context switches require a kernel mode switch.

---

## PART 2: Systems Programming API Reference

### 1. `fork()` — Duplicate Calling Process

**Header:** `<unistd.h>`

```
pid_t fork(void);
```

- **How it Operates:** Duplicates the calling process to create a child clone. The child gets a copy of the parent's data, stack, heap, and file descriptors.
- **Execution Flow:** The CPU does not restart execution at `main()`. The kernel duplicates the Program Counter (PC) at the exact moment of the call, and both processes resume execution at the instruction immediately following `fork()`.
- **Return Values:**
  - To the Child: Returns `0` (used as a flag meaning "you are the child"; the child can call `getpid()` to find its actual PID).
  - To the Parent: Returns the Child's Real PID ( `> 0` ), allowing the parent to track, signal, or wait for that specific child.
  - On Failure: Returns `-1` (system out of memory or process limit reached).
- **Formulas:** A series of `n` consecutive `fork()` calls without conditions produces  $2^n$  concurrent processes.

## 2. `wait()` — Wait for Process Termination

**Header:** `<sys/wait.h>`

```
pid_t wait(int *wstatus);
```

- **Purpose:** Suspends execution of the parent process until one of its child processes terminates.
- **Arguments:** `int *wstatus` points to an integer to store termination status flags. Pass `NULL` if exit flags are unneeded.
- **Process Lifecycle Artifacts:**
  - **Zombie Process:** A child process that has completed execution, but its parent has not called `wait()`, leaving an entry in the kernel's process table.
  - **Orphan Process:** A child process whose parent exited without waiting; automatically adopted by `init/systemd` (PID 1).

## 3. `execvp()` — Execute a File / Program Image Replacement

**Header:** `<unistd.h>`

```
int execvp(const char *file, char *const argv[]);
```

- **Behavior:** Replaces the current process image entirely with a new executable program. On success, it never returns because the original code is overwritten.
- **Arguments:**

- `file` : Name of the executable (e.g., `"ls"` ); automatically searched across directories in the `$PATH` environment variable.
- `argv` : Array of string pointers representing arguments. `argv[0]` is the program name; the final array element must always be `NULL` .
- **Return Value:** Returns `-1` only on failure.

## 4. `strtok()` — String Tokenizer

**Header:** `<string.h>`

```
char *strtok(char *str, const char *delim);
```

- **Behavior:** Splits a string into tokens based on specified delimiter characters. It replaces each delimiter character with a null terminator ( `'\0'` ) in memory and returns a pointer to the start of the token.
- **Usage Pattern:** Pass the source string on the first call; pass `NULL` on subsequent calls to continue parsing the same buffer. Returns `NULL` when no tokens remain.

## 5. `fgets()` & `strcspn()` — Input Stream Reading & Stripping

**Headers:** `<stdio.h>` , `<string.h>`

```
char *fgets(char *s, int size, FILE *stream);
size_t strcspn(const char *s, const char *reject);
```

- `fgets()` reads a line from stdin including spaces until a newline or EOF (Ctrl + D) is reached. Returns `NULL` on EOF.
- `input[strcspn(input, "\n")] = '\0';` finds the index of the trailing `\n` stored by `fgets` and replaces it with a null terminator.

## 6. `perror()` & `exit()` — Systems Error Handling

**Headers:** `<stdio.h>` , `<stdlib.h>`

```
void perror(const char *s);
void exit(int status);
```

- `perror()` : Reads the global `errno` code set by a failed system call and prints `"<s>: <OS description human-readable>\n"` .

- `exit(status)` : Terminates the calling process immediately. `0` indicates success; non-zero values (like `1`) indicate errors.
- 

## PART 3: Milestone 1 — Implementation & Bug Post-Mortem

### The Core Shell Loop

The shell executes an infinite loop through five sequential stages:

Prompt → Read (`fgets`) → Parse (`strtok`) → Fork → Exec / Wait

Print Prompt

`fgets()` + Parse

`fork()`

`[id==0]`

`[id > 0]`

`execvp()`

`wait(NULL)`

`perror()`

(Parent waits for child)

`exit(1)`

### Post-Mortem: Bugs Encountered & Solutions

#### 1. Token Skipping via Double `strtok` Invocations

- *The Bug*: Calling `strtok(NULL, " ")` inside the `while` loop condition **and** inside the loop body caused every second argument to be consumed and lost.
- *The Fix*: Call `strtok` only once per iteration, store the pointer in `arg[i]`, and advance `i`.

## 2. Missing NULL Termination & Uninitialized Memory

- *The Bug:* `char *arg[64]` allocated on the stack contains random garbage values. Iterating until `arg[i] != NULL` read past the last token into uninitialized pointers, causing invalid memory dereferences and Segmentation Faults.
- *The Fix:* Explicitly assign `arg[i] = NULL;` immediately after the parsing loop finishes.

## 3. Duplicate Parent Processes on `execvp` Failure

- *The Bug:* If `execvp()` failed (e.g., non-existent command like `pwe`), execution continued past the `if (id == 0)` block. The child process looped back to the top of `while (1)`, resulting in two parent shells reading from the keyboard simultaneously.
- *The Fix:* Place `perror("habishell"); exit(1);` immediately after `execvp()` to ensure the failed child terminates instantly and returns control to the waiting parent.

## 4. Edge Cases: Empty Enter & Ctrl + D (EOF)

- *The Bug:* Pressing Enter sets `arg[0] = NULL`, causing `execvp(NULL, arg)` to segfault. Pressing Ctrl + D caused `fgets()` to return `NULL`, resulting in infinite loops over stale buffer memory.
- *The Fix:*
  - Add `if (fgets(...) == NULL) break;` to gracefully exit the shell on Ctrl + D.
  - Add `if (arg[0] == NULL) continue;` to safely re-prompt when Enter is pressed.